

TP4 - Indexation de Texte par Arbres Binaires de Recherche

NF16 - A25

4 décembre 2025

1 Introduction

Ce rapport présente l'implémentation d'un système d'indexation de texte utilisant les arbres binaires de recherche (ABR). Le projet permet de charger un fichier texte, d'indexer tous ses mots avec leurs positions, et de reconstruire le texte à partir de l'index.

2 Structures et Fonctions Supplémentaires

2.1 Structures Supplémentaires

En plus des structures demandées (`T_Position`, `T_Noeud`, `T_Index`), nous avons implémenté une structure auxiliaire :

- **Structure Phrase** : Utilisée pour les questions B.6 et B.7, elle facilite la reconstruction des phrases à partir de l'index. Elle contient :
 - `numeroPhrase` : identifiant de la phrase
 - `mots` : tableau dynamique de pointeurs vers les mots
 - `ordres` : tableau des ordres correspondants
 - `taille` et `capacite` : gestion dynamique de la taille

Justification : Cette structure permet d'éviter des parcours répétés de l'ABR lors de la reconstruction des phrases. Elle optimise les opérations B.6 et B.7 en centralisant les données nécessaires.

2.2 Fonctions Auxiliaires

- `initialiserIndex()` : Initialise un index vide
- `convertirMinuscules()` : Convertit un mot en minuscules pour ignorer la casse
- `libererPositions()`, `libererNoeud()`, `libererIndex()` : Gestion de la mémoire
- `afficherNoeudInfixe()` : Affichage récursif de l'index
- `collecterPhrasesRécursif()` : Collecte toutes les phrases de l'index
- `ajouterMotPhrase()` : Ajoute un mot à une phrase avec réallocation dynamique

Justification : Ces fonctions assurent la modularité du code, facilitent la maintenance et garantissent une gestion correcte de la mémoire dynamique.

3 Complexité des Fonctions

3.1 Fonctions de Base

1. **ajouterPosition()** : $O(p)$ où p est le nombre de positions dans la liste. La fonction parcourt la liste pour insérer la position au bon endroit tout en maintenant l'ordre.
2. **ajouterOccurrence()** : $O(h + p)$ où h est la hauteur de l'arbre et p le nombre de positions du mot. Dans le pire cas (arbre dégénéré) : $O(n + p)$ où n est le nombre de mots distincts. En moyenne (arbre équilibré) : $O(\log n + p)$.
3. **indexerFichier()** : $O(m \times (h + p))$ où m est le nombre de mots dans le fichier. Chaque mot nécessite une insertion dans l'ABR. Dans le meilleur cas (arbre équilibré) : $O(m \times \log n)$.
4. **afficherIndex()** : $O(n \times p)$ où n est le nombre de mots distincts et p le nombre moyen de positions. Le parcours infixe visite chaque nœud une fois et affiche toutes ses positions.
5. **rechercherMot()** : $O(h)$ où h est la hauteur de l'arbre. Dans le meilleur cas : $O(\log n)$, dans le pire cas : $O(n)$.
6. **afficherOccurrencesMot()** : $O(n \times p + k \times m \log m)$ où :
 - n = nombre de mots distincts
 - p = nombre moyen de positions par mot
 - k = nombre d'occurrences du mot cherché
 - m = nombre moyen de mots par phrase

La fonction collecte toutes les phrases puis trie les mots de chaque phrase concernée.

7. **construireTexte()** : $O(n \times p + s \log s + s \times m \log m)$ où :
 - n = nombre de mots distincts
 - p = nombre moyen de positions
 - s = nombre de phrases
 - m = nombre moyen de mots par phrase

La fonction collecte toutes les phrases, les trie, puis trie les mots dans chaque phrase.

3.2 Fonctions Auxiliaires

- **convertirMinuscules()** : $O(k)$ où k est la longueur du mot
- **libererIndex()** : $O(n)$ où n est le nombre total de nœuds
- **collecterPhrasesRecuratif()** : $O(n \times p)$
- **ajouterMotPhrase()** : $O(1)$ amorti (réallocation avec doublement de capacité)

4 Choix d'Implémentation et Optimisations

4.1 Gestion de la Casse

Tous les mots sont convertis en minuscules lors de l'insertion dans l'ABR. Cela permet de traiter "Voiture", "voiture" et "VOITURE" comme un seul mot, conformément aux exigences. La conversion est effectuée par la fonction **convertirMinuscules()**.

4.2 Ordre Lexicographique

L'utilisation de **strcmp()** garantit un ordre alphabétique naturel dans l'ABR. Cette fonction est optimisée en C et assure des comparaisons cohérentes.

4.3 Liste Triée de Positions

Les positions sont maintenues triées par (ligne, ordre) dans une liste chaînée. Bien que l'insertion soit en $O(p)$, cela simplifie grandement l'affichage et la reconstruction du texte, évitant des tris ultérieurs coûteux.

4.4 Optimisation Mémoire

- Utilisation de tableaux dynamiques avec doublement de capacité pour éviter des réallocations fréquentes
- Libération systématique de la mémoire dans l'option 7 du menu
- Pas de duplication inutile des chaînes de caractères

5 Conclusion

L'implémentation proposée respecte toutes les exigences du TP tout en privilégiant la simplicité et l'efficacité. Les choix algorithmiques permettent d'obtenir des complexités raisonnables pour des textes de taille moyenne. Les structures auxiliaires, bien que non demandées, améliorent significativement les performances des fonctions de reconstruction de phrases et de texte.

Les principales forces de cette implémentation sont :

- Code simple et lisible
- Gestion rigoureuse de la mémoire dynamique
- Complexités optimales pour un ABR non équilibré
- Modularité facilitant la maintenance

Une amélioration possible serait l'implémentation d'un ABR auto-équilibré (AVL ou Rouge-Noir) pour garantir des complexités logarithmiques même dans le pire cas.